SIMULAZIONE DELLA GESTIONE DELLE EMERGENZE DI UNA CITTÀ TRAMITE ARCHITETTURA DHT CHORD

Elena Battistoni

Corso di Laurea Triennale in Statistica per l'Economia e l'Impresa
Sistemi di Elaborazione 2

Anno accademico 2025/2026
Padova, 11 marzo 2026

1. INTRODUZIONE

La gestione delle emergenze, che nei contesti urbani moderni sono generate ininterrottamente, richiede qualità sempre più sofisticate da parte delle infrastrutture informatiche a supporto della rete fisica, come velocità, efficienza ed affidabilità.

Questo progetto, che modella il problema reale come una simulazione ad eventi discreti, intende mostrare come un'architettura ad eventi, integrata con una rete DHT basata sul protocollo Chord e adattata al contesto della gestione delle emergenze, possa offrire una soluzione fluida e resiliente alla questione.

Si vuole quindi testare un'architettura a conoscenza decentralizzata, in cui l'instradamento e la risoluzione delle criticità sono autonome.

2. STATO DELL'ARTE

Al fine di testare la risposta delle infrastrutture informatiche senza costi importanti, la letteratura scientifica identifica come soluzione lo standard metodologico nella Simulazione ad Eventi Discreti (DES), teorizzata da Banks e Carson (1986). Con questo metodo il tempo di simulazione avanza tramite salti discreti da un evento a quello successivo, senza tenere conto degli intervalli di inattività del sistema (Next-Event Time Advance).

Passando al dominio della gestione delle emergenze, è di ispirazione il lavoro di Mecella et al. (2006) che propongono WORKPAD, un'infrastruttura peer-to-peer adattiva per il coordinamento di operatori umani in scenari di disastro, evidenziando come un approccio distribuito sia più resiliente rispetto ai sistemi centralizzati tradizionali. Il presente progetto si colloca in questa direzione, adottando specificamente DHT Chord come meccanismo di instradamento delle richieste di intervento.

Il protocollo Chord, introdotto da Stoica et al. (2003), definisce un'architettura DHT in cui i nodi sono organizzati su un ring circolare e il routing avviene tramite finger table con complessità O(log N). Originariamente concepito per applicazioni peer-to-peer di file sharing, Chord è stato studiato come base per sistemi distribuiti che richiedono lookup efficienti in assenza di coordinamento centralizzato.

3. SOLUZIONE ADOTTATA

Il sistema progettato simula la gestione di eventi critici, generati nei diversi quartieri di una città fittizia, grazie ad un'architettura suddivisa in tre livelli principali.

Il primo livello è la città simulata, composta da 10 quartieri: Centro Storico, Borgo Nuovo, Borgo Antico, Industriale, Residenziale, Est, Ovest, Colli Bassi, Colli Alti, Montagna. Ciascun quartiere genera emergenze casuali e indipendenti in modo autonomo, e ciò viene eseguito sfruttando una distribuzione di Poisson con parametro differente per quartiere: l'utilizzo di questa distribuzione probabilistica è giustificato dal fatto che gli eventi sono discreti, indipendenti tra loro e generati in un intervallo di tempo definito (quello in cui la simulazione avanza).

Il secondo livello invece è la rete di gestione delle emergenze, implementata come una DHT Chord e popolata da nodi di intervento specializzati. Le specializzazioni possibili implementate nel progetto sono: ambulanza, vigili del fuoco, polizia, protezione civile e soccorso alpino.

Le emergenze, trasformate in richieste di intervento specifiche e codificate tramite un target ID, vengono instradate nella DHT tramite il nodo più vicino al target; da qui, tramite le finger table, cercheranno un nodo compatibile in grado di prendersi carico della richiesta.

Quando nessun nodo è immediatamente disponibile, la richiesta viene inserita in una coda ordinata in base alla gravità dell'emergenza: verrà ripresa non appena un nodo adeguato si libererà.
Il terzo livello è il sistema di gestione delle emergenze, che permette la costruzione della città, la gestione della simulazione e la redazione del report finale.
Nell'architettura ad eventi progettata, questo livello funge da event manager, permettendo ai due livelli descritti in precedenza di comunicare tra loro senza avere conoscenza l'uno di ciò che fa l'altro. La loro comunicazione è quindi asincrona e mediata: la città e la rete interagiscono esclusivamente attraverso una lista di emergenze ordinata temporalmente (secondo *tp_emergenza)*. È proprio questa lista a costituire il meccanismo centrale della simulazione ad eventi discreti: il tempo non scorre in modo continuo, ma avanza di evento in evento.

4. STRUMENTI TECNICI

Il progetto è stato interamente sviluppato in linguaggio Python.
Le librerie utilizzate sono tutte parte della libreria standard di Python, e sono:

- *random*: utilizzata per la generazione delle emergenze nel singolo quartiere. In particolare *random.expovariate* modella gli intervalli di tempo tra un'emergenza e la successiva secondo una distribuzione esponenziale (coerentemente con il processo di Poisson adottato), mentre *random.choices* seleziona il tipo di emergenza da generare utilizzando come pesi le probabilità definite per ciascun quartiere.
- *csv*: utilizzata per la lettura dei file che definiscono la struttura della città e della rete di gestione delle emergenze. I file sono: *quartieri.csv* (che configura ciascun quartiere della città), *emergenze.csv* (il quale contiene la probabilità che ogni possibile emergenza avvenga in ciascun quartiere della città) e *nodi_intervento.csv* (che definisce i nodi di intervento della città).
- *time*: questa libreria è stata utilizzata per la gestione delle pause nell'interfaccia testuale, rendendo la simulazione osservabile in modo realistico.

Il progetto è suddiviso in quattro moduli: *citta_simulazione.py*, che definisce i quartieri; *struttura_intervento.py*, che implementa la DHT e i nodi di intervento; *sistema_emergenze.py*, che coordina la simulazione; *interfaccia.py*, la quale gestisce la visualizzazione testuale del progetto (su riga di comando).

5. DESCRIZIONE IN DETTAGLIO

5.1 LA CITTÀ E LE EMERGENZE

Ogni quartiere della città è modellato dalla classe *Quartiere* ed è caratterizzato da un nome, un identificatore di posizione (*posizione_id*, da 0 a 9) e un parametro lambda (*lam*) di intensità, che rappresenta il tasso medio di eventi che si verificano nel quartiere. I valori di lambda variano per quartiere: mentre CentroStorico possiede il valore più alto (2.5), Montagna ha quello più basso (0.4).
La pianificazione e la generazione della prossima emergenza sono effettuate con i metodi *pianifica_emergenza* e *generazione_emergenza* della classe *Quartiere*, e il momento in cui la crisi avverrà viene salvato nell'attributo *tp_emergenza* del quartiere.
Le emergenze invece sono definite dalla classe *M_Emergenza*, che caratterizza l'evento per quartiere di provenienza, descrizione, momento in cui avviene e gravità, un valore intero campionato uniformemente tra 1 e 3, il quale influenzerà le risorse necessarie per gestire la crisi e la sua durata stimata. La probabilità che un

determinato tipo di emergenza si possa verificare è differente da quartiere a quartiere, coerentemente con la natura di ciascuno: ad esempio nel quartiere Industriale dominano incendi e fughe di gas, mentre in Montagna prevalgono persone disperse e incidenti escursionistici.

In seguito sono elencati tutti i possibili tipi di emergenza che si possono verificare durante la simulazione, suddivisi per specializzazione:

- ambulanza: infarto, caduta accidentale, intossicazione o avvelenamento, emergenza ostetrica, crisi epilettica;
- polizia: rapina in corso, controllo traffico e incidenti, gestione manifestazioni o proteste, scomparsa persona, truffa o frode;
- protezione civile: alluvione o esondazione, frana o smottamento, terremoto, evacuazione di zona a rischio, blackout elettrico diffuso;
- soccorso alpino: persona dispersa in montagna, recupero in zone difficili, emergenza nevosa o ghiaccio, incidente escursionistico;
- vigili del fuoco: incendio edificio, incendio veicolo, incendio bosco, fuga gas, crollo strutturale;

Ogni emergenza generata nel quartiere verrà memorizzata nel suo storico: questa informazione tornerà utile nella fase di report.

5.2 LA RETE DI GESTIONE DELLE EMERGENZE

5.2.1 LE RICHIESTE DI EMERGENZA E GLI ID

Prima che le emergenze create nei quartieri entrino nella rete DHT per essere prese in carico dai nodi di intervento, queste sono trasformate in richieste di intervento dalla classe *RichiestaIntervento*, che interpreta ciascuna crisi e la rende gestibile dalla rete; il procedimento consiste nel reinterpretare le informazioni contenute nel messaggio di emergenza ricavandone la specializzazione richiesta (derivata dalla descrizione dell'emergenza tramite un dizionario), le risorse necessarie (calcolate moltiplicando un valore base per la gravità dell'emergenza) e la durata stimata dell'intervento (anche questa proporzionale alla gravità). Viene inoltre calcolato il *target_id*, che diventerà il punto di ingresso ideale della richiesta nella rete DHT. Questo meccanismo di identificazione è alla base dell'adattamento del protocollo Chord al contesto della gestione delle emergenze nel progetto: infatti sia i nodi di intervento che le richieste di intervento sono codificati con un ID avente la stessa struttura, rendendo così possibile la corrispondenza diretta tra emergenza e nodo in grado di gestirla.

Gli ID sono calcolati seguendo una struttura "Specializzazione-Unità-Quartiere". Quelli delle richieste di intervento, calcolati tramite la funzione *calcolo_target_id*, presentano la cifra delle centinaia che si riferisce alla specializzazione richiesta per risolvere l'emergenza (codificata nel seguente modo: ambulanza = 0, vigili del fuoco = 1, polizia = 2, protezione civile = 3, soccorso alpino = 4), la cifra delle decine che rappresenta il numero di unità di intervento richieste (da 1 a 9), e la cifra delle unità che indica la *posizione_id* del quartiere da cui la richiesta di intervento è partita.

Allo stesso modo gli ID dei nodi di intervento, calcolati in *generazione_id*, sono scomponibili in specializzazione che il nodo offre, numero di unità di intervento messe a disposizione (sempre da 1 a 9) e *posizione_id* del quartiere in cui il nodo è localizzato.

Lo spazio degli indirizzi del ring ha dimensione 2^m con m = 9 (che permette quindi 512 posizioni). Poiché il valore massimo di ID raggiungibile secondo la codifica appena descritta è 499, tutti gli ID ricadono naturalmente nello spazio senza necessità di riduzione.

5.2.2 FINGER TABLE E ROUTING

La rete è implementata come un ring circolare Chord di dimensione $2^9 = 512$ tramite le classi *Ring* e *DHT*, dove i nodi sono ordinati per ID crescente. Ogni nodo mantiene una finger table, gestita dalla classe *FT*, che contiene m = 9 riferimenti ad altri nodi del ring: l'i-esimo elemento della finger table del nodo con ID n punta al primo nodo con ID maggiore o uguale a $n+2^i$.

La ricerca del nodo competente avviene tramite il metodo *search* della classe *DHT*: la richiesta entra nella rete dal nodo più vicino al *target_id* e viene instradata tramite *searchNewId*, che seleziona dalla finger table il nodo più vicino al target senza superarlo. Se il nodo raggiunto non è compatibile o è occupato, viene generato un nuovo target incrementando le unità richieste di 1 (di conseguenza *target_id* aumenterà di 10), cercando così un nodo con più risorse disponibili. La ricerca termina quando viene trovato un nodo compatibile e libero, oppure quando sono state vagliate tutte le alternative possibili senza trovare un nodo adatto.

5.3 SISTEMA EMERGENZE

5.3.1 GESTIONE DELLA SIMULAZIONE

La simulazione è coordinata dalla classe *SistemaEmergenze*.

Il ciclo principale è gestito dal metodo *simulazione*, che riceve come parametro il numero di emergenze da generare. Ad ogni iterazione il sistema identifica il quartiere con il *tp_emergenza* più basso, ovvero quello che genererà la prossima emergenza, e avanza il tempo di simulazione a quel momento; prima di generare la nuova emergenza, si controlla se eventi precedentemente iniziati si sono conclusi, e in caso positivo le risorse occupate vengono rilasciate e le code vengono controllate. Solo successivamente viene generata la nuova emergenza, trasformata in richiesta di intervento e instradata nella rete DHT.

Se la ricerca ha successo, il nodo trovato alloca le risorse e viene registrato il momento previsto in cui queste torneranno disponibili; se invece nessun nodo è disponibile, la richiesta è inserita nella coda della specializzazione corrispondente.

Una volta raggiunto il numero di emergenze richiesto dall'input, il ciclo continua fino a quando tutti gli interventi in corso non sono terminati e le code sono state processate.

5.3.2 GESTIONE DELLE CODE

Come spiegato in precedenza, se nella ricerca non risulta disponibile nessun nodo compatibile, la richiesta di intervento viene inserita nella coda della specializzazione corrispondente: è infatti mantenuta una coda separata per ciascuna delle cinque specializzazioni, all'interno del dizionario *code_attesa*. Le richieste in coda sono ordinate per gravità decrescente, garantendo così che le emergenze più critiche vengano evase per prime.

Le code vengono controllate dal metodo *controlla_coda* ogni volta che un nodo rilascia delle risorse; ad ogni controllo viene ritentato l'inserimento nella DHT della richiesta in testa alla coda.

Il controllo viene effettuato solo sulla coda della specializzazione del nodo appena liberato, ottimizzando il numero di ricerche.

5.3.3 REPORT FINALE

Al termine della simulazione, la classe *Report* raccoglie e analizza i dati prodotti, costruendo un report.

A partire dalle informazioni date dagli storici di quartieri e nodi e dai dati registrati da *SistemaEmergenze* su stress e salti, le informazioni prodotte sono:

- la distribuzione delle emergenze per quartiere;
- la distribuzione delle emergenze per specializzazione (ottenuta aggregando gli storici dei nodi di intervento);
- i cinque nodi più operativi (quelli che hanno gestito più interventi);
- lo stato delle code al termine della simulazione;
- lo studio del carico di rete, che comprende l'analisi dello stress della DHT (calcolato come la percentuale di nodi occupati) e dei salti effettuati tra nodi nelle ricerche avvenute durante la simulazione.

Quest'ultima informazione diventa rilevante nell'analisi dell'efficacia del routing: una media di salti bassa indica che la rete ha operato in condizioni ottimali, mentre valori elevati segnalano saturazione e degradazione delle prestazioni.

5.4 INTERFACCIA

L'interfaccia del progetto, grazie alla quale l'utente è in grado di interagire con il programma, utilizza le funzioni definite in *interfaccia.py*. Queste convergono nella funzione *main*, dove è costruito il menu principale: esso presenta 4 opzioni, l'ultima delle quali permette la chiusura del programma.

Scegliendo l'opzione 1 "Guarda la città" si possono osservare i 10 quartieri della città, ognuno accompagnato dal conteggio di nodi di intervento presenti nella zona, divisi per specializzazione.

L'opzione 2 "Fai partire la simulazione" permette l'avvio della simulazione con delle personalizzazioni: si può infatti selezionare la velocità con cui si visualizzano le emergenze (con pause o senza) e scegliere quante farne partire. Nel momento in cui la simulazione è avviata, compariranno sulla console gli eventi generati: le emergenze sono evidenziate in giallo, le allocazioni riuscite in verde, i fallimenti in rosso, i rilasci di risorse in azzurro e i recuperi dalla coda in viola; per ciascun evento vengono inoltre mostrati i dettagli tecnici della richiesta e il percorso nella DHT.

Infine l'opzione 3 "Vedi report finale", disponibile dopo aver fatto partire la simulazione, permette la visualizzazione del report, che nella sezione successiva verrà analizzato nel dettaglio.

6. RISULTATI EMPIRICI

Si analizza ora una simulazione effettuata con 150 eventi, ritenuti sufficienti per dare significatività statistica al report finale.

La configurazione della città, già spiegata in precedenza, comprende 10 quartieri e 50 nodi di intervento (10 per specializzazione), e rimane invariata durante la simulazione.

Terminata la simulazione, si genera il report finale, riportato nella figura seguente.

```
REPORT CONCLUSIVO DELLA SIMULAZIONE

NUMERO DI EVENTI SIMULATI: 150

DISTRIBUZIONE EMERGENZE PER QUARTIERE
CentroStorico  | (27)
BorgoAntico    | (27)
Est            | (19)
BorgoNuovo     | (17)
Residenziale   | (17)
Industriale    | (15)
ColliBassi     | (11)
ColliAlti      | (8)
Ovest          | (6)
Montagna       | (3)

DISTRIBUZIONE EMERGENZE PER SPECIALIZZAZIONE
🚓 Polizia           | (51)
🚑 Ambulanza         | (45)
🔥 Vigili_fuoco      | (29)
🛡️ Protezione_civile | (22)
🧗 Soccorso_alpino   | (3)

NODI DI INTERVENTO PIÙ OPERATIVI
1. Nodo 18  [AMBULANZA      ] in ColliAlti    -> 7 interventi
2. Nodo 26  [AMBULANZA      ] in Ovest        -> 7 interventi
3. Nodo 253 [POLIZIA        ] in Industriale  -> 7 interventi
4. Nodo 217 [POLIZIA        ] in ColliBassi   -> 6 interventi
5. Nodo 226 [POLIZIA        ] in Ovest        -> 6 interventi

STATO DELLE CODE AL TERMINE DELLA SIMULAZIONE
✔ Tutte le emergenze sono state gestite correttamente.

ANALISI CARICO DI RETE E PICCHI
STRESS DI PICCO : [                                        ] 64.0%
STRESS MEDIO    : [                                        ] 42.7%

EFFICIENZA DEL ROUTING (HOP COUNT)
Media salti per ricerca: 5.11
Salto più lungo        : 14 nodi
Valutazione rete       : Ottima (Ricerca efficiente)
```

Si nota che la distribuzione delle emergenze per quartiere rispecchia i parametri lambda assegnati in configurazione: infatti CentroStorico, con valore di lambda più alto (2.5), si conferma il quartiere con il maggior numero di eventi (27), mentre Montagna e Ovest, con i parametri lambda minori (rispettivamente 0.4 e 0.6) registrano valori più contenuti (3 e 6).

Per quanto riguarda la distribuzione per specializzazione, polizia e ambulanza dominano (con rispettivamente 51 e 45 interventi), mentre il numero minore di emergenze generate è di soccorso alpino (3): il valore basso è giustificabile con la natura geografica limitata delle emergenze di questo tipo.

La predominanza di emergenze di tipo ambulanza e polizia è evidente anche guardando ai nodi di intervento più operativi, che appartengono proprio alle specializzazioni citate; da questa classifica si comprende anche che la rete è in grado di distribuire il carico in modo bilanciato, visto che il numero di interventi effettuati dai primi 5 nodi più attivi risulta essere pressoché lo stesso (6-7).

Al termine della simulazione tutte le code sono state gestite correttamente, e risultano quindi vuote: questo indica che la rete dispone di risorse sufficienti ad assorbire l'intero carico.

Lo stress medio della rete (la media delle percentuali di nodi occupati registrate in diversi istanti della simulazione) è risultato pari al 42.7%, con un picco massimo del 64%: la rete ha quindi operato senza mai raggiungere la saturazione totale.

La media dei salti per ricerca risulta essere 5.11, con un massimo di 14 nodi attraversati.

Per interpretare correttamente questi valori è necessario precisare che il contatore dei salti accumula tutte le iterazioni del ciclo di ricerca per una stessa richiesta (quindi comprende anche i tentativi successivi al primo, che comportano la modifica del target).

Con 50 nodi nel ring, una singola ricerca compie $\log_2 50 \approx 6$ salti. Poiché il contatore accumula tutti i tentativi successivi e il numero massimo di questi è 8 (U incrementabile da 1 a 9), il worst case teorico è $8 \cdot \log_2 50 \approx 45$ salti; il massimo empiricamente osservato di 14 salti è ampiamente inferiore a questo limite, il che indica che nella pratica ogni tentativo costa molto meno di una ricerca completa: infatti il nodo corrente si trova già in una posizione avanzata nel ring, e il nuovo target dista poco da esso.

Si nota che il valore massimo di salti (14) è un limite costante e quindi indipendente dal numero di eventi simulati: poiché nodi e finger table sono fissi, si presume che il percorso peggiore sia sempre identico.

La complessità della ricerca rimane in ogni caso O(log N), con il fattore introdotto dalle modifiche del target che, essendo costante, non modifica la complessità. In questo caso, con una media di 5.11 salti, la valutazione complessiva della rete è ottima.

7. COMMENTO CRITICO

Il sistema progettato dimostra che un'architettura DHT Chord è applicabile con efficacia in un ambito non convenzionale come quello della gestione delle emergenze urbane: infatti l'adattamento del protocollo tramite la codifica degli ID nella forma "SUQ" consente una corrispondenza diretta e naturale tra richieste e nodi, e i risultati empirici confermano che la rete opera in modo efficiente e bilanciato.

Un limite attuale del sistema riguarda la gestione dei nodi occupati: quando un nodo viene trovato ma risulta impegnato, viene incrementato U senza considerare la possibilità di cercare un nodo equivalente in un quartiere diverso, portando a tentativi che potrebbero essere evitati. Un altro limite del progetto ad ora riguarda l'assunzione per cui ogni nodo può gestire una sola emergenza alla volta: questo aspetto infatti, sebbene semplifichi l'implementazione, non rispecchia pienamente la realtà.

Possibili sviluppi futuri comprendono l'introduzione della possibilità per ogni nodo di gestire più operazioni contemporaneamente (svincolandosi dall'attributo booleano e sfruttando il numero di unità disponibili), l'introduzione di emergenze che richiedono più specializzazioni (dovendole quindi coordinare contemporaneamente), e la possibilità di rendere la rete più dinamica (permettendo l'aggiunta e la rimozione di nodi durante la simulazione).

8. BIBLIOGRAFIA

Banks, J., & Carson, J. S. (1986). Introduction to discrete-event simulation. *Proceedings of the 18th Conference on Winter Simulation*, 17–23. https://doi.org/10.1145/318242.318253

Mecella, M., Angelaccio, M., Krek, A., Catarci, T., Buttarazzi, B. & Dustdar, S. (2006). WORKPAD: an Adaptive Peer-to-Peer Software Infrastructure for Supporting Collaborative Work of Human Operators in Emergency/Disaster Scenarios. *IEEE Conference Publication*. https://ieeexplore.ieee.org/abstract/document/1644131

Stoica, I., Morris, R., Liben-Nowell, D., Karger, Kaashoek, M., Dabek, F., & Balakrishnan, H. (2003). Chord: a scalable peer-to-peer lookup protocol for internet applications. *IEEE/ACM Transactions on Networking*, *11*(1), 17–32. https://doi.org/10.1109/tnet.2002.808407